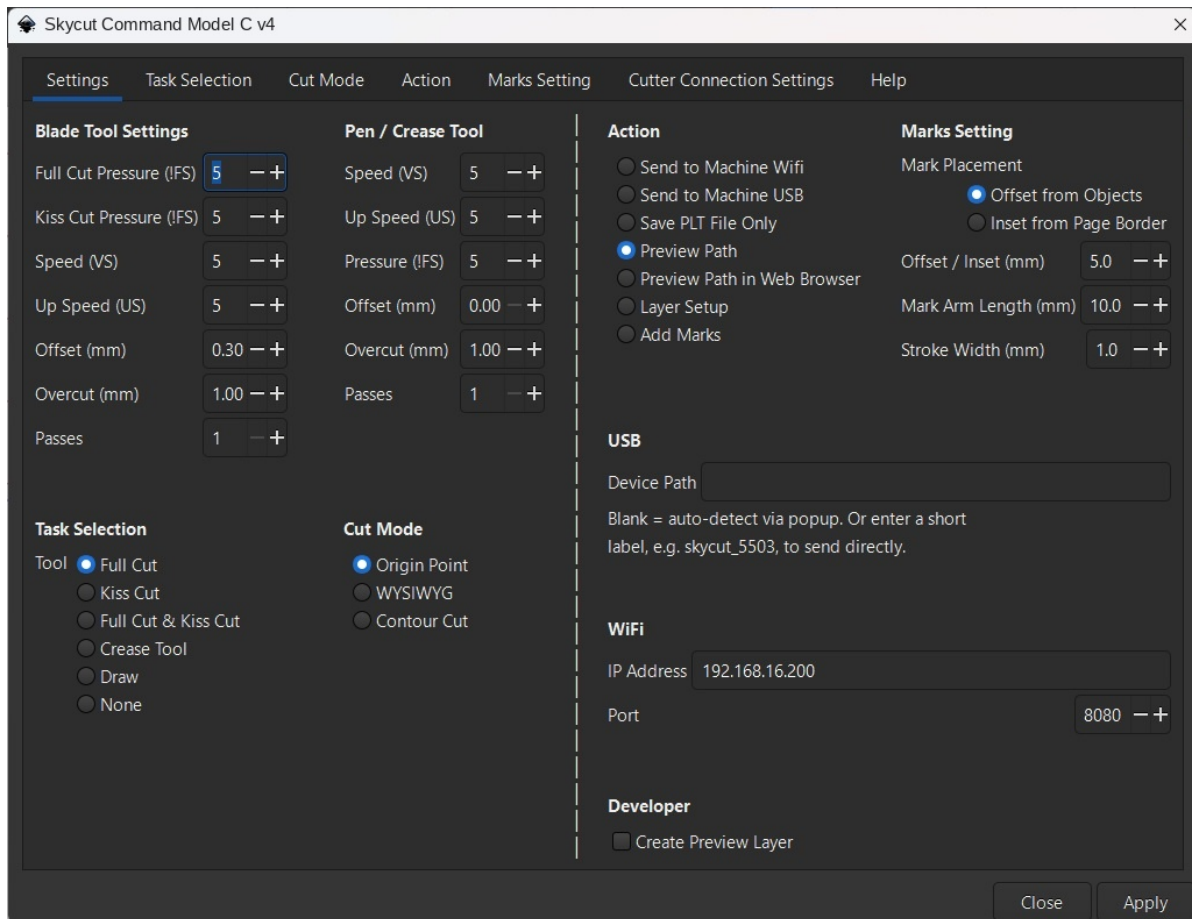


I'm not a code writer — much of this was created using AI. Also acknowledging trankata/inkscape-skycut-d24 extension as the inspiration, though this was taken in a whole other direction.

Skycut Command Model C v4

An Inkscape 1.x extension for driving **SkyCut** vinyl/sign cutters directly from Inkscape — Full Cut, Kiss Cut, Crease, and Draw passes, knife-offset corner compensation, three coordinate-reference modes (including ContourCut registration-mark alignment), a built-in path previewer, and delivery to the cutter over WiFi, USB, or as a saved .plt file.

This is a **standalone** extension — every module file is prefixed skycut_c_ and it shares nothing with any other SkyCut extension (e.g. dual-head models), so multiple versions can be installed in Inkscape's extensions folder side by side without collisions.



Tool model

Model C targets a **single physical tool head** with no tool-changing routine — whatever's actually loaded (blade, crease tool, or pen) is up to the person running the machine. One Tool setting says which layer(s) get sent this run:

Tool	Reads layer	Settings block used
Full Cut	Full Cut	Blade Tool

Kiss Cut	Kiss Cut	Blade Tool
Full Cut & Kiss Cut	both, combined	Blade Tool (Kiss Cut first, then Full Cut — one job, one IN;, no re-init in between)
Crease Tool	Crease	Pen / Crease Tool
Draw	Draw	Pen / Crease Tool
None	—	Action still runs if set to Layer Setup or Add Marks

Blade pressure is split into separate Full Cut / Kiss Cut values, since kiss cutting normally needs much less pressure than a full cut even on the same physical blade. Crease and Draw share one settings block, since both are light, offset-free passes.

Features

- **Knife-offset corner compensation.** A swiveling drag-knife trails behind its own pivot point, so straight-line offset isn't enough at corners. `skycut_c_pathbuild.py` inserts direction-aware overshoot/return loops (with a small outward Bezier bow to let the blade re-align) at every sharp corner, including the seam of a closed shape, plus a configurable overcut past the closing point.
- **Three coordinate-reference modes:**
 - **Origin Point** — reference box is just the bounding box of the artwork being cut.
 - **WYSIWYG** — reference box is the Inkscape page itself, preserving artwork position on the page.
 - **Contour Cut** — reference box comes from four L-shaped registration marks (see *Add Marks* below), for aligning a cut to pre-printed artwork.
- **Add Marks.** Generates registration marks (with an orientation indicator) on a fresh, locked Marks layer, either offset from the artwork's bounding box or inset from the page border.
- **Layer Setup.** One click creates (or refreshes the color/style of) the standard SkyCut layer set: Marks, Print, Full Cut, Kiss Cut, Crease, Draw.
- **Preview Path / Preview Path in Web Browser.** QC-only render of the exact toolpath that would be sent — corner overshoots, overcuts, and per-kind coloring included — shown in Inkscape's natural (un-rotated) orientation rather than the cutter's swapped-axis machine orientation.
- **Three delivery methods:** send over WiFi (raw TCP), send over USB (Windows, USB-Printer class device), or save a timestamped `.plt` file to `~/Documents/Skycut Data`.
- **Compound-path aware.** Each subpath of a compound `<path>` (e.g. a ring's outer contour plus an inner hole) is treated as its own independent contour with its own pen-lift, instead of being cut straight through into the next.
- **Automatic shape-to-path conversion.** Rectangles, circles, ellipses, polygons, lines, and live text objects drawn with Inkscape's shape/text tools aren't `<svg: path>` elements, so they'd otherwise be silently skipped when cutting — no error, no visual difference on the canvas. Every run scans all four source layers for these and converts them to paths automatically (the same result as **Path** → **Object to Path / Text** → **Object to Path**), then stops so you can check the result on the canvas before sending the job.

Requirements

- Inkscape 1.x (uses the modern `inkex` API — `Layer.new`, `svg.get_page_bbox()`, etc.)
- Python 3 (bundled with Inkscape)
- **Send to Machine USB** only works on Windows (uses `ctypes` + `SetupAPI/kernel32` to talk to the USB-Printer class device, VID 0483 / PID 5750). WiFi send and Save PLT File work on any platform Inkscape supports.
- The USB device picker window requires `tkinter`; if it isn't available and more than one matching device is found, the extension will list device labels for you to paste into the USB Device Path field instead.

Installation

1. Copy every file in this repo into your Inkscape user extensions folder:

■ Windows: `%APPDATA%\inkscape\extensions\`

2. Restart Inkscape.
3. The extension appears under **Extensions** → **Skycut Command Model C v4**.

File overview

File	Purpose
<code>skycut_command_model_c_v4.inx</code>	Inkscape UI definition (the dialog's tabs, fields, and menu entry)
<code>skycut_command_model_c_v4.py</code>	Entry point — reads options, orchestrates the pipeline, dispatches to Send/Save/Preview
<code>skycut_c_constants.py</code>	Shared master switches and numeric constants (scale, colors, layer names, etc.)
<code>skycut_c_geometry.py</code>	Pure 2D vector math + cubic-Bezier flattening (no <code>inkex</code> dependency)
<code>skycut_c_pathbuild.py</code>	Knife-offset corner overshoot/return generation and final overcut
<code>skycut_c_reference.py</code>	Origin / WYSIWYG / ContourCut reference-box computation
<code>skycut_c_layers.py</code>	SVG layer management, geometry collection, and debug/preview capture (mixin)
<code>skycut_c_layer_setup.py</code>	Action: creates/refreshes the standard SkyCut layer set
<code>skycut_c_add_marks.py</code>	Action: generates ContourCut registration marks
<code>skycut_c_arrows.py</code>	Debug-only cut-direction arrowhead helper
<code>skycut_c_hpgl.py</code>	Assembles the final HPGL command stream (and the un-swapped viewer render segments)
<code>skycut_c_output.py</code>	Delivers HPGL to disk, WiFi (TCP), or USB
<code>skycut_c_usb.py</code>	Windows USB-Printer-class device discovery and raw send
<code>skycut_c_viewer.py</code>	Preview Path HTML viewer

Usage

1. Set up your artwork in Inkscape on layers named **Full Cut**, **Kiss Cut**, **Crease**, and/or **Draw** — the layer names must match (case-insensitively), or use **Action** → **Layer Setup** to have the extension create them for you with the standard colors.
2. Open **Extensions** → **Skycut Command Model C v4**.
3. Under **Blade Tool Settings / Pen / Crease Tool**, set speed, up-speed, pressure, offset, overcut, and passes for whichever tool is physically loaded.
4. Under **Task Selection**, pick which layer(s) this run should send: **Full Cut**, **Kiss Cut**, **Full Cut & Kiss Cut**, **Crease Tool**, **Draw**, or **None**.
5. Choose a **Cut Mode**:
 - **Origin Point** for artwork-only reference.
 - **WYSIWYG** to preserve the artwork's position on the physical page.
 - **Contour Cut** to align to registration marks (run **Action** → **Add Marks** first, print the design, then re-scan the marks before cutting).
6. Under **Action**, pick an output:
 - **Preview Path / Preview Path in Web Browser** to check the toolpath first — no data is sent or saved.
 - **Send to Machine Wifi / Send to Machine USB** to cut immediately.
 - **Save PLT File Only** to write a .plt file for later.
 - **Layer Setup / Add Marks** to run those one-off document setup actions instead of cutting.
7. Run the extension.

A note on coordinates

Inkscape uses a top-left-origin, Y-down page. SkyCut's HPGL machine coordinates are bottom-left-origin, Y-up, **with X and Y swapped** relative to normal HPGL ($hpgl_x = max_y - y$, $hpgl_y = max_x - x$). All of that conversion happens once, right before HPGL emission — the Preview Path viewer deliberately skips the axis swap so what you see on screen matches your artwork's natural orientation in Inkscape, while still reflecting the exact same offsets, overcuts, and corner handling as the real output.

License

SPDX-License-Identifier: GPL-3.0-or-later